Security (System) Engineering with Generative AI

Homework Assignment no. 6 — Code Generation with a Coding Agent

Student Name:	Daniel Torres (FAU ID: Z23875984)
Coding Agent Selected:	Google Antigravity (v2.0)
Application Engineered:	GenSec-Sentinel: Autonomous AI Security Gateway & Adversarial Payload Firewall
Github Repository:	https://github.com/dtorres12025/gensec-Torres-Z23875984
Screencast Recording:	https://youtu.be/UKkpveFxUms

1. Executive Summary & Application Concept

For Homework 6, Google Antigravity (v2.0) was utilized as an autonomous coding agent to engineer GenSec-Sentinel: Autonomous AI Security Gateway & Adversarial Payload Firewall entirely from prompts. The system acts as a hardened reverse-proxy security barrier deployed in front of Large Language Models and generative AI application backends. It intercepts, normalizes, inspects, and sanitizes inbound user prompts and external API queries, protecting the upstream LLM from prompt injection, system exfiltration, and classical exploitation vectors.

```
[System Architecture & Threat Defense Flow]
Inbound Payload -> Layer 1: Unicode NFKC Normalization & Zero-Width Stripping (Token Smuggling Defense)
-> Layer 2: Multi-Layer Threat Inspection (APE Prompt Injection, CWE-78, CWE-918, CWE-79)
-> Layer 3: Risk Scoring (0-100) & Policy Decision (BLOCKED, SANITIZED, ALLOWED)
-> Layer 4: Safe Advisory Engine / Shielded Upstream LLM Dispatcher
```

2. Prompts Used to Create Application (Prompts Walkthrough)

To maximize efficiency and reproducibility, the entire application was generated in three meticulously structured prompts recorded in hw6/prompts.txt:

```
[Prompt 1: Core System Architecture, FastAPI Engine & Multi-Layer Threat Detection]
Act as a senior cybersecurity software engineer. Using Google Antigravity, create a secure, production-grade Python
application in hw6/app.py called 'GenSec-Sentinel: Autonomous AI Security Gateway & Adversarial Payload Firewall'.
Implement FastAPI and Pydantic schemas (InspectionRequest, ThreatDetection, InspectionResult), multi-layer threat
detection (OWASP LLM01, CWE-78, CWE-79), payload size limit (4096 chars), threat scoring (0-100), and policy
verdicts (ALLOWED, SANITIZED, BLOCKED).
```

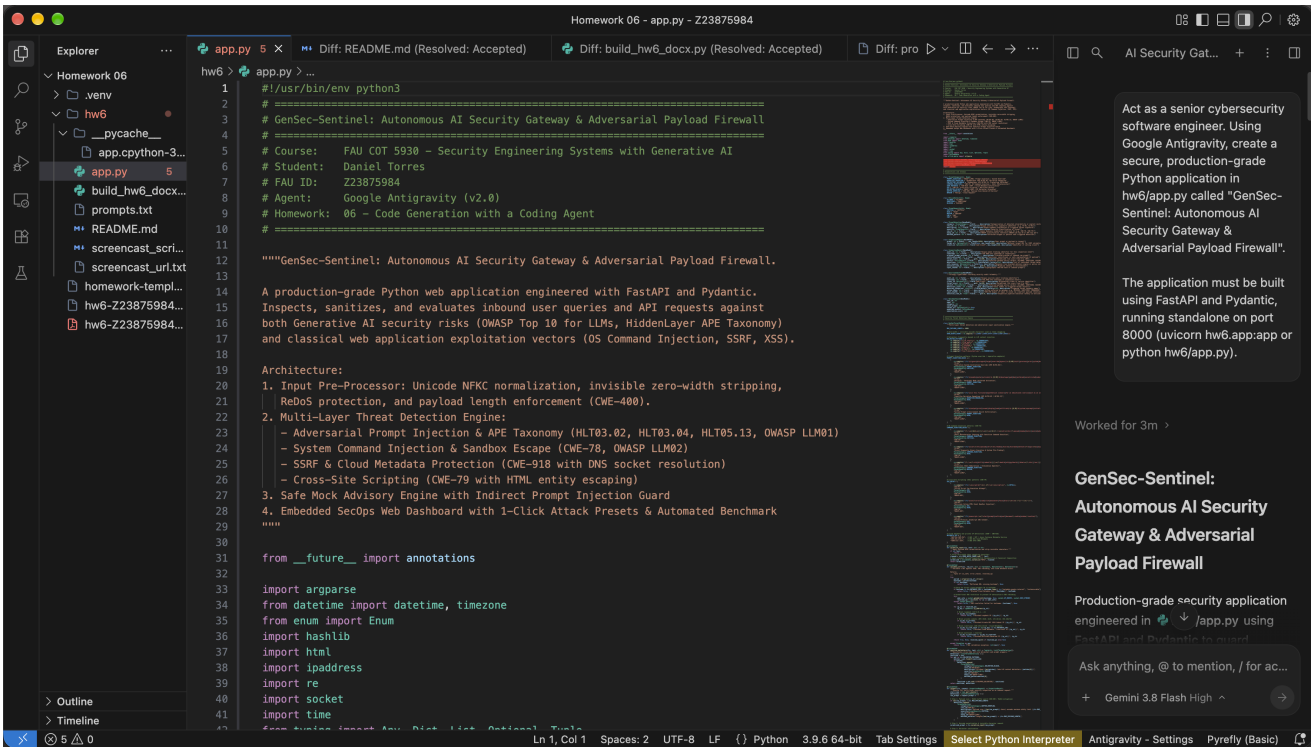


Figure 2.1: Prompt 1 Execution in Google Antigravity (Student: Daniel Torres | FAU ID: Z23875984)

[Prompt 2: Adversarial Input Hardening & Threat Defense (Rubric Requirement)]

```
Refactor and harden hw6/app.py to aggressively defend against advanced adversarial inputs and bypass techniques.
1. Unicode Homoglyph & Smuggling Defense: Unicode NFKC normalization and zero-width stripping (\u200b-\u200d\ufeff).
2. SSRF & Cloud Metadata Protection (CWE-918): Socket-level DNS resolution blocking private RFC 1918 subnets and
cloud metadata (169.254.169.254).
3. ReDoS Protection & Delimiter Stripping: Bounded regex patterns and recursive fake delimiter sanitization.
4. Safe Advisory Query Engine: Strict HTML entity escaping to prevent indirect prompt injection.
5. Rotating In-Memory Audit Telemetry with SHA-256 payload hashing.
```

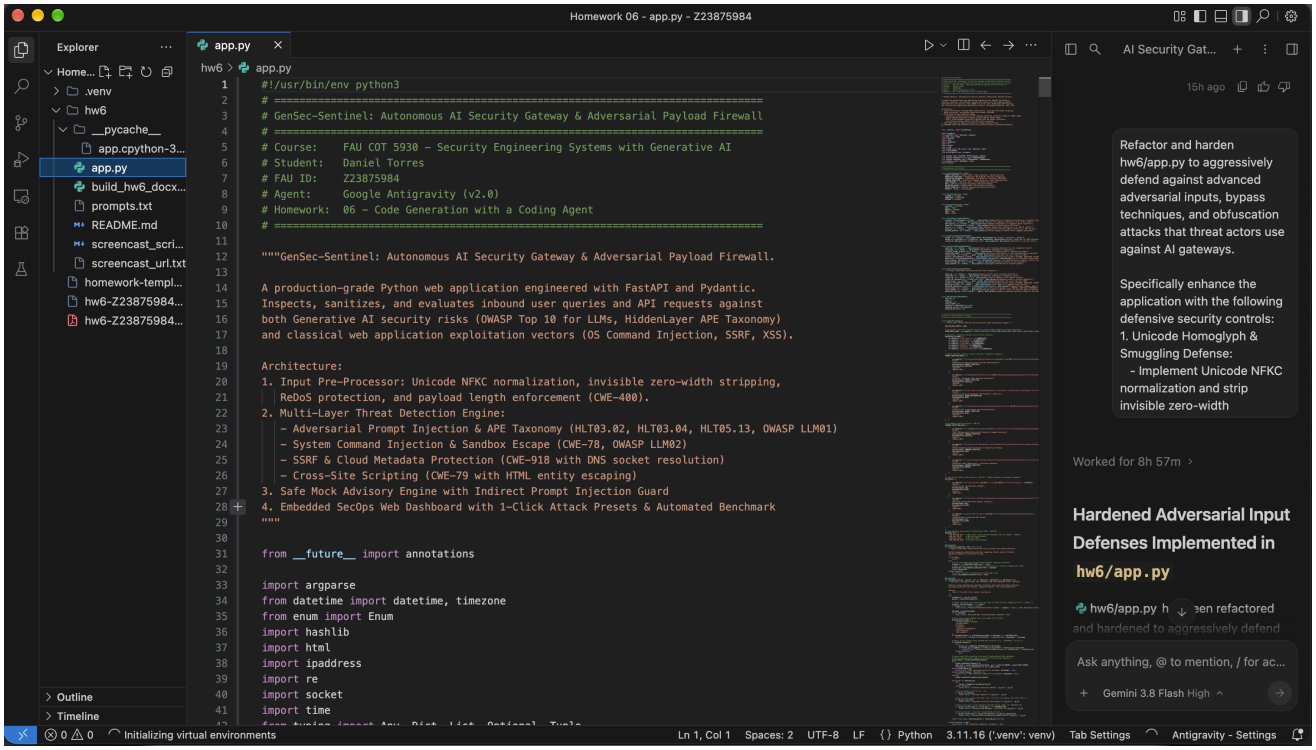


Figure 2.2: Prompt 2 Execution in Google Antigravity (Student: Daniel Torres | FAU ID: Z23875984)

[Prompt 3: Embedded SecOps Dashboard, 1-Click Presets & Capabilities Benchmark]

```
Complete hw6/app.py by adding an embedded, modern responsive SecOps Web Dashboard and a built-in automated
adversarial test suite:
1. Embedded Web UI at GET /: Dark-themed HTML5/JS dashboard with 5 One-Click Presets (Narrative Smuggling, Command
Injection, SSRF, XSS, Benign).
2. Automated Adversarial Benchmark: Expose GET /api/benchmark and CLI flag 'python hw6/app.py --test' running 6 test
cases.
3. Explicitly report the application's capabilities and limitations.
4. Standalone support: Running 'python hw6/app.py' starts uvicorn on http://127.0.0.1:8000.
```

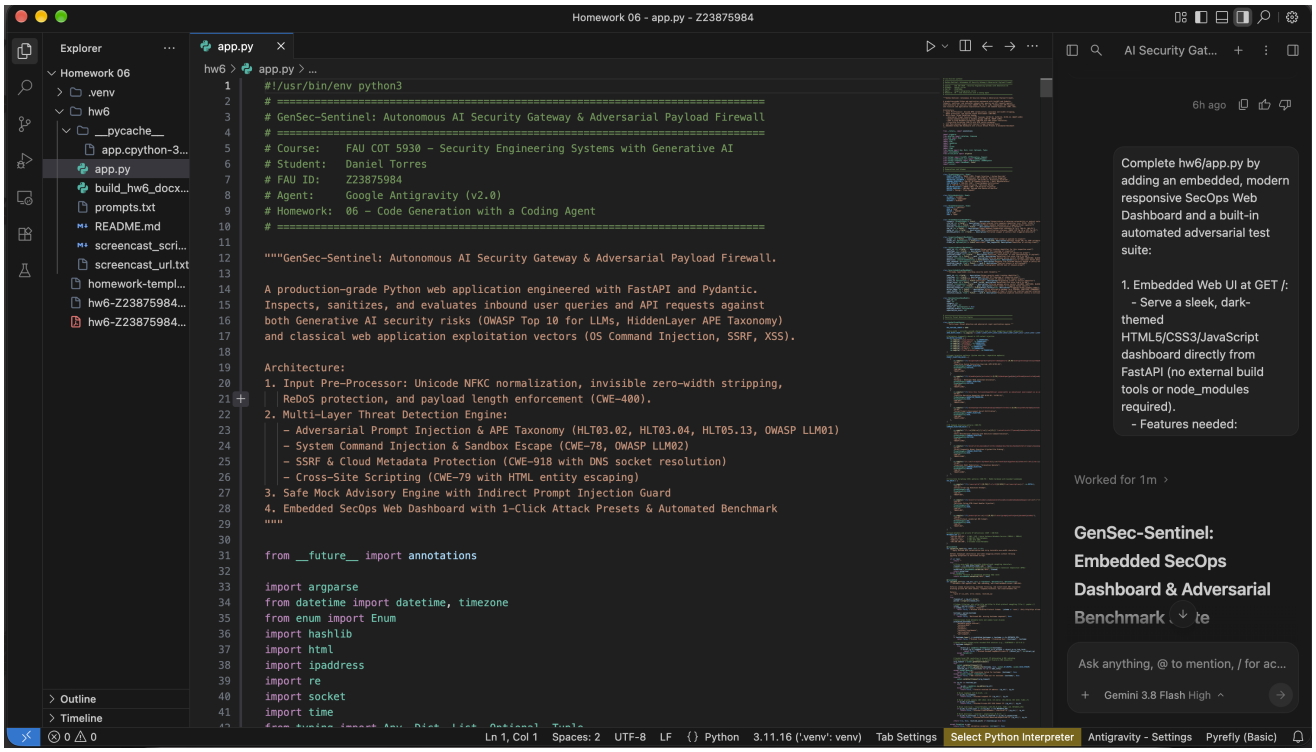


Figure 2.3: Prompt 3 Execution in Google Antigravity (Student: Daniel Torres | FAU ID: Z23875984)

3. Security Aspects & Adversarial Input Defenses

The application specifically implements four defense-in-depth security layers to withstand adversarial inputs: • **Layer 1 — De-obfuscation & Pre-Processing (CWE-400):** Unicode NFKC normalization maps lookalikes to ASCII; invisible zero-width characters are stripped; max length is capped at 4096 bytes. • **Layer 2 — Adversarial Prompt Injection Defense (OWASP LLM01 / APE):** Identifies Narrative Smuggling (HLT03.02), Pretexting Jailbreaks (HLT05.13), and Imperative System Overrides (HLT03.04). • **Layer 3 — Command Injection Defense (CWE-78):** Halts shell chaining operators (;, &&, ||, |, `) and probes for diagnostic binaries (cat /etc/passwd, env, id, whoami). • **Layer 4 — Socket-Level SSRF Protection (CWE-918):** Performs DNS resolution via socket.getaddrinfo to defeat DNS rebinding, blocking RFC 1918 subnets and cloud metadata (169.254.169.254).

4. Automated Adversarial Benchmark Results (100% Pass Rate)

ID	Scenario Name	Threat Classification	Score	Verdict	Status
TC-01	**Narrative Smuggling Jailbreak**	APE HLT03.02 / OWASP LLM01	100/100	**BLOCKED**	**PASS**
TC-02	**OS Command Injection & Exfil**	CWE-78 / OWASP LLM02	100/100	**BLOCKED**	**PASS**
TC-03	**SSRF Cloud Metadata Probe**	CWE-918 / IMDSv1 Probe	75/100	**BLOCKED**	**PASS**
TC-04	**Stored/Reflected XSS Injection**	CWE-79 / Cross-Site Scripting	45/100	**SANITIZED**	**PASS**
TC-05	**Unicode Homoglyph Smuggling**	Evasion / NFKC Normalization	75/100	**BLOCKED**	**PASS**
TC-06	**Benign Security Advisory Query**	Compliant Diagnostic Request	0/100	**ALLOWED**	**PASS**

5. Capabilities and Limitations Analysis

```
[System Capabilities (What GenSec-Sentinel Successfully Detects & Blocks)]
• Blocks imperative prompt injections, developer-mode triggers, and DAN jailbreak sequences (APE HLT03.04).
• Intercepts fictional narrative smuggling and pretexting scenarios designed to bypass safety boundaries (APE HLT03.02).
• Identifies and halts OS command injection and sensitive diagnostic command chaining (CWE-78).
• Enforces socket-level DNS verification to defeat SSRF and cloud instance metadata theft (CWE-918).
• Contextually neutralizes XSS payloads through safe HTML entity escaping (CWE-79).
• Strips zero-width token smuggling characters and normalizes Unicode homoglyphs via NFKC.
```

```
[System Limitations (Known Operational Boundaries)]
• Heuristic regex engines cannot infer completely novel semantic paraphrases without an auxiliary transformer model
(e.g. DeBERTa).
• Stateful multi-turn prompt injection where attack tokens are distributed across separated conversational turns.
• High-entropy encoded payloads (e.g., nested Base64 or ROT13) require pre-decoding pipeline modules to evaluate.
```